

APPLICATION SECURITY ASSESSMENT

OctoSniff Local Authorization and Party Chat Review

Login-state bypass, PlayStation add-on bypass, offline Community Filters reconstruction, and username/IP correlation analysis

Assessment date: July 26-27, 2026

Assessed build: OctoSniff 5.1.1 for Windows

Authorization: Owner-authorized local assessment

Target SHA-256: 2CF7F902FEA4E2109210CA36BE8ADE859B0CB7FB1ACEE4794B34AC08D785A317

Overall risk: CRITICAL - paid-feature authorization is enforced in an untrusted client

Bottom line The Party Chat subscription gate can be bypassed locally. The PoC removes the OctoSniff login requirement, opens Party Chat in an account-free offline diagnostic state, restores the 37-name Community Filters catalog, and installs exact local PSN Party and Tunnel packet rules. A separate owner-authorized linked-PSN run also reached the live Party workflow. Obfuscation slowed inspection but did not preserve the authorization boundary after execution.

Executive summary

OctoSniff 5.1.1 performs the decisive PlayStation add-on checks in its Svelte/WebView frontend and Go application backend. Both are under the local user's control. Runtime extraction recovered the frontend bundle and named Go functions, allowing a build-specific launcher to force authenticated state, force the frontend entitlement result, and replace the native entitlement guards with successful returns.

The tested unentitled account originally received the PlayStation add-on purchase screen. Under the PoC, Party Chat appeared as an unlocked navigation item and opened the Puller. After the owner linked a controlled PSN account, the patched process loaded live parties and member rosters. A controlled decryption run completed and left the party automatically. It returned no address because the other test member was marked Server Relay, which makes a direct peer IP unavailable.

The more important architectural issue is the observed Party VPS protocol. The client constructs requests with a PSN access token and a shared application credential reconstructed from embedded values. No subscriber-specific OctoSniff capability was identified in the Party gateway request path. If the VPS has no additional unseen control, it cannot independently distinguish a paid subscriber from a locally patched client.

An authenticated owner-authorized catalog request returned 37 Community Filter summaries. PSN Party and PSN Tunnel included complete rule bodies and were reproduced offline using the application's local custom-filter engine. Those definitions match UDP protocol, port, packet-length and payload signatures; they are packet classifiers, not a username-decryption cipher.

ID	Severity	Finding
F-01	Critical	Party Chat paid authorization is client-controlled
F-02	Critical	Party VPS lacks an observed subscriber-bound authorization assertion
F-03	Medium	Local authentication state can be forced without completing login
F-04	High	Embedded shared Party gateway credential is recoverable at runtime
F-05	Medium	Obfuscated application code and symbols are recoverable after launch
F-06	Low	Executable is unsigned and carries minimal version metadata
F-07	Medium	Account-backed Community Filters can be reconstructed locally

Scope and methodology

- Installed Windows client and its local runtime behavior.
- Login-state, current-user and PlayStation add-on entitlement checks.
- Party Chat Puller navigation, live PSN party enumeration and a controlled decryption run.
- Party gateway client construction, credentials, encryption helpers and request flow.
- PlayStation username/IP correlation paths exposed by the client.
- Authenticated Community Filter catalog enumeration and exact PSN Party/Tunnel rule reconstruction.
- No destructive changes to the installed Program Files copy; all bypasses were process-memory only.

Methods included binary fingerprinting, signature inspection, mapped-memory recovery, Go pcIntab parsing, targeted disassembly, frontend logic review, controlled runtime patching, UI validation, and protocol-path analysis.

Safety boundary The PoC is tied to the assessed SHA-256, launches only its bundled local copy, refuses unknown builds, redirects runtime state to package-local APPDATA and LOCALAPPDATA paths, and serves offline data only on IPv4 loopback. It does not expose PSN tokens, OctoSniff credentials, email addresses, shared gateway keys, or captured member data.

Verified proof of concept

The launcher starts an isolated copy suspended, waits for the protected image to unpack, patches the frontend before normal rendering, waits for relocated Go symbols, and then applies five native patches. It verifies each write and logs the original and replacement bytes. The installed executable remains unchanged.

1. Frontend authentication bootstrap: create an immediate local PoC profile without an OctoSniff account lookup or login page.
2. Frontend logout bridge and logout transition: prevent a return to the login wall.
3. Frontend entitlement result: change the playstation_addon fallback from false to true.
4. Frontend capability/paywall map: expose locally gated navigation without depending on a paid profile.
5. Native authentication helpers: return true.
6. Native hasSubEntitlement: return true so adapter enumeration and capture are not rejected by the general subscription guard.
7. Native hasPlayStationAddon: return true.
8. Native requirePlayStationAddon: return a nil error.
9. Network initialization: prefer a physical default or active adapter, and use an active virtual adapter only as a fallback.
10. Offline catalog: load all 37 recovered Community Filter names without an OctoSniff account; mark 35 incomplete entries metadata-only.
11. Exact PSN rules: install and toggle PSN Party and PSN Tunnel through the existing local custom-filter engine.
12. Account-free Party Chat diagnostic: present the real Party Chat interface with an empty Offline PoC party state and no persisted PSN credential.

Observed result The final isolated build cold-started without an OctoSniff account, displayed Party Chat without a lock or PSN-link redirect, loaded 37 offline Community Filter entries, and installed two exact PSN packet rules. Removing PSN Party through the UI reduced the local rule list from two to one; restoring it returned the list to two. The default-interface fallback also eliminated the empty-selection state.

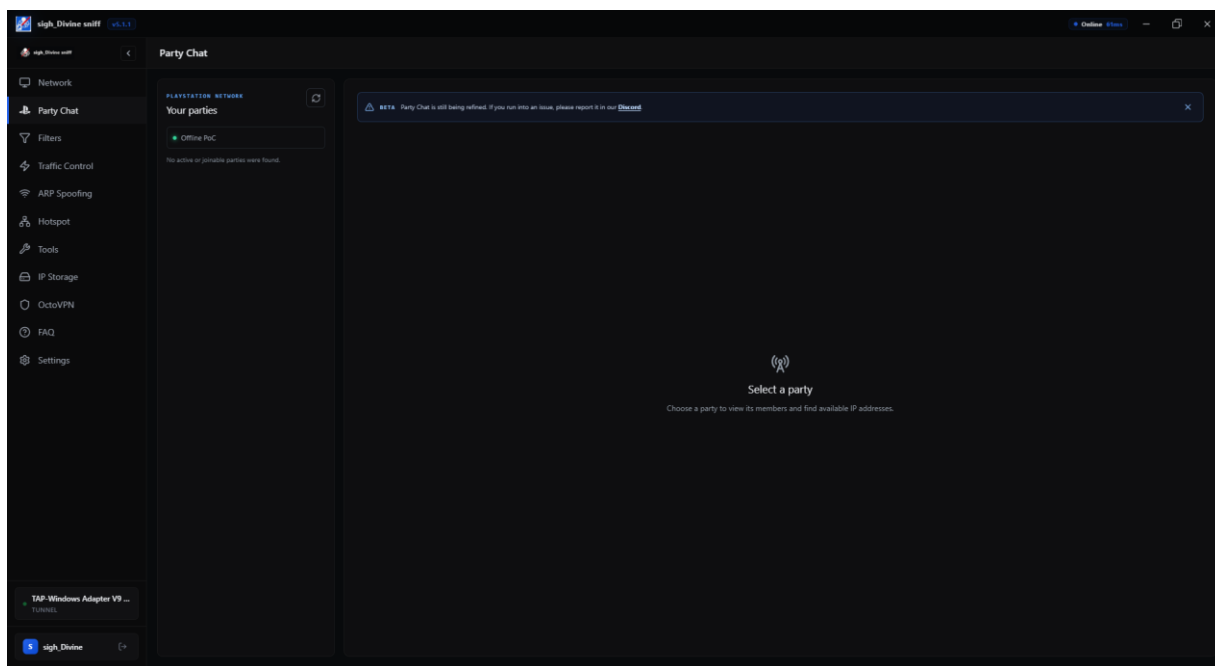


Figure 1. Party Chat reached without an OctoSniff or PSN account in the final isolated Offline PoC state.

Live controlled-party result

The owner linked a PSN account inside the isolated PoC and made a controlled party available. The Puller successfully listed live parties and member rosters. Selecting the two-member test party exposed one other player and enabled Decrypt IPs. The action completed, reported that the party was left automatically, and returned zero IPs because the other participant used PSN Server Relay.

Interpretation This is a successful end-to-end execution of the bypassed feature with a protected/no-address outcome. Server Relay means the client communicates with Sony relay infrastructure instead of the other console, so there is no direct peer address for the Puller to discover.

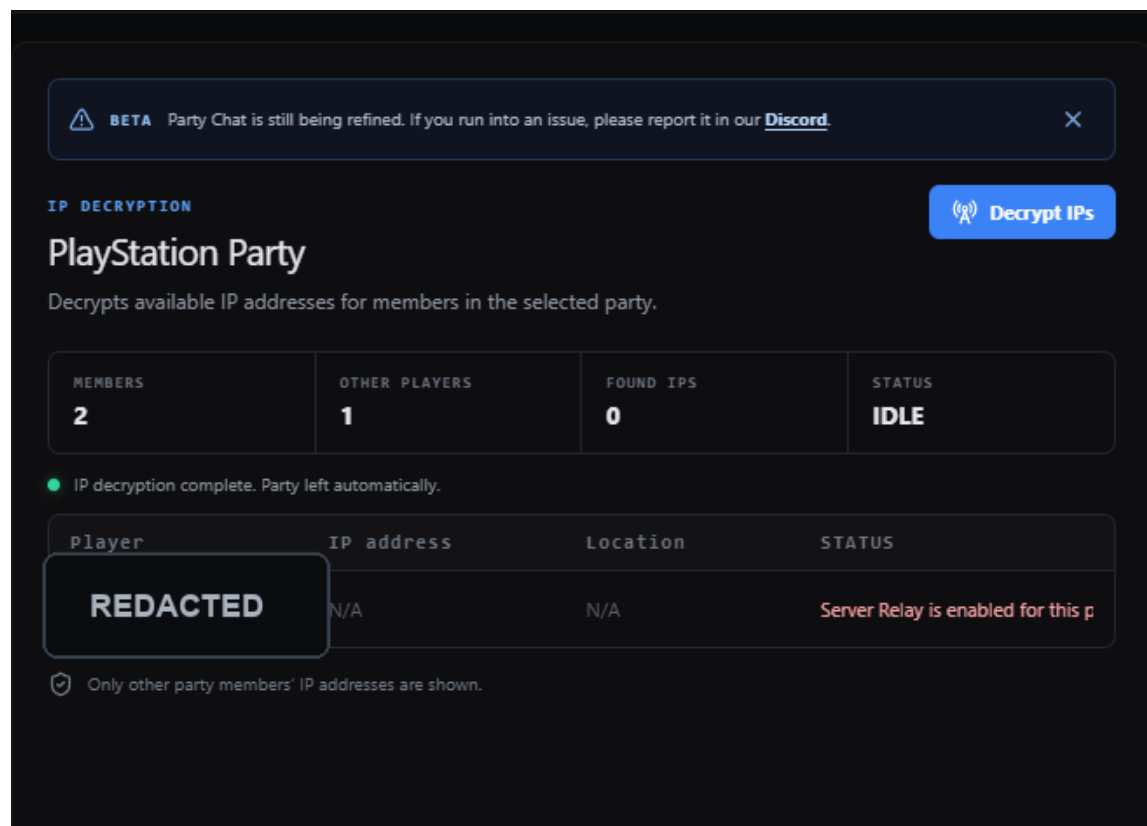


Figure 2. Controlled decryption completed with zero addresses because Server Relay was enabled. The player name was redacted; no IP was present.

Offline Community Filters reconstruction

The authenticated catalog contained 37 unique filter identifiers. The final PoC serves those summaries from 127.0.0.1 and does not call the account-backed catalog API. PSN Party and PSN Tunnel start enabled and are mirrored into the application's local filter store. The remaining 35 cards are metadata-only and deliberately cannot install guessed rules.

Filter	Protocol / ports	Length	Payload signatures	Role
PSN Party	UDP 1024-65535	96-140	2 exact prefixes	P2P
PSN Tunnel	UDP 50000-65535	96-158	6 exact prefixes	Server

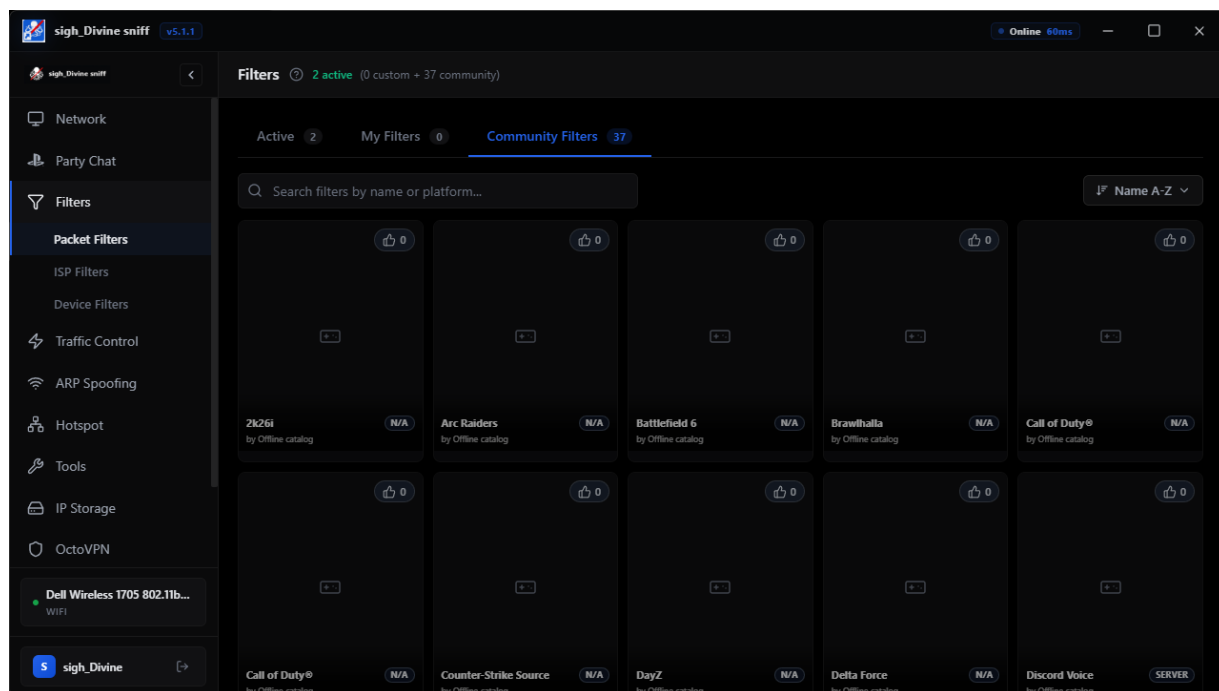
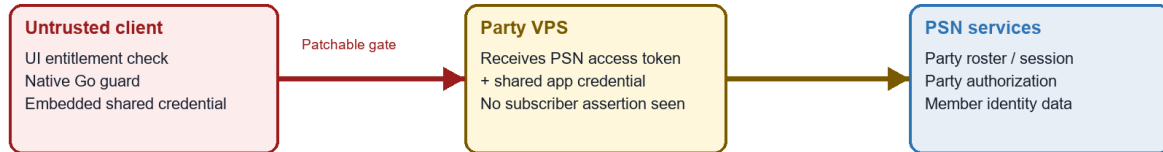


Figure 3. The final no-account PoC restores the 37-entry Community Filters catalog; PSN Party and Tunnel carry exact local rules.

Authorization architecture

Current versus required Party Chat authorization boundary

CURRENT



REQUIRED

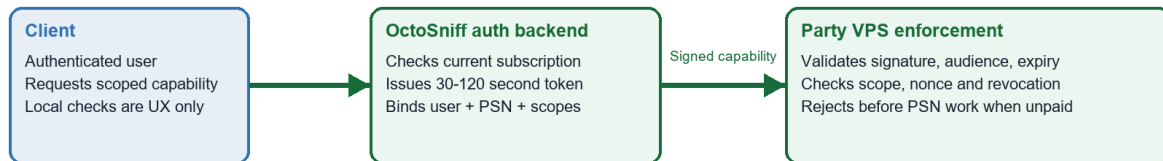


Figure 4. Current observed trust boundary and the recommended subscriber-bound capability design.

Obfuscation, C++ rewrites, anti-debugging and certificate pinning may increase effort, but none can make a local boolean or embedded shared secret authoritative. The paid decision must be made by a service that the user cannot patch.

Detailed findings

F-01 Party Chat paid authorization is client-controlled **CRITICAL**

The frontend derives access by searching user entitlements for the exact lowercase string `playstation_addon`. The native backend repeats the decision in `hasPlayStationAddon` and `requirePlayStationAddon`, while the broader `hasSubEntitlement` guard also protects adapter enumeration and capture. All of these checks execute in memory owned by the local user.

- Frontend gate changed with a one-byte false-to-true modification.
- `hasSubEntitlement` was replaced with a true return, which also restored interface enumeration in the PoC.
- `hasPlayStationAddon` was replaced with a true return.
- `requirePlayStationAddon` was replaced with a nil-error return.
- The Party Chat Puller opened for an account whose recovered entitlement graph did not contain `playstation_addon`.

Impact A user can unlock the monthly Party Chat interface without purchasing the add-on and can reach the PSN-dependent workflow.

F-02 Party VPS lacks an observed subscriber-bound authorization assertion **CRITICAL**

The Party request context checks the local entitlement, builds the embedded Party gateway client, and obtains a valid PSN Party access token. The gateway client constructor accepts the session-service URL, PSN access credential, key identifiers, public encryption material and a shared authentication key. No OctoSniff user token or short-lived paid-feature capability was identified in this path.

- Observed remote origin: `https://s0ny-sniff.octosniff.net`.
- Observed Party submission route string: `/party/submit`.
- Embedded configuration includes Party encryption/authentication key IDs and a recoverable shared 32-byte authentication value.
- The gateway can authenticate the application build or request envelope, but the observed material does not prove the OctoSniff account has a current subscription.

Inference Unless the VPS performs an additional control not represented in the client protocol, it will accept a patched client that presents the embedded app credential and a valid PSN token. This inference is strongly supported by the recovered call path but should be confirmed server-side during remediation.

F-03 Local authentication state can be forced without completing login **MEDIUM**

The application shell relies on local `IsAuthenticated` results and a frontend authentication-state variable. The PoC replaces the startup flow with an immediate local profile, performs no OctoSniff account lookup, neutralizes logout-to-login transitions, and enters the authenticated shell without cached profile metadata.

Important distinction This does not mint a valid OctoSniff backend session. Server APIs that independently validate a session token may still reject the client. The finding is a local shell/state bypass, not proof that backend account authentication is broken.

F-04 Embedded shared Party gateway credential is recoverable **HIGH**

The Party client reconstructs its authentication key locally. Environment-variable hooks and embedded-value reconstruction are both visible after runtime recovery. Any credential distributed to every client must be treated as public and cannot be the sole authorization factor for a paid service.

- Rotate the current Party gateway authentication credential after the server-side subscriber-token change.
- Do not place a replacement long-lived licensing secret in the executable.

F-05 Runtime recovery defeats the protection boundary **MEDIUM**

The on-disk PE is heavily packed and high entropy, which blocks casual static inspection. After launch, however, the mapped module exposes the complete Svelte bundle, Go `pcIntab`, function names and executable code. Stable names revealed the entitlement and Party gateway functions directly.

- Recovered frontend size: 1,660,445 bytes.
- Recovered Go function table: approximately 23,700 entries, varying slightly with initialization timing.
- Examples include hasPlayStationAddon, requirePlayStationAddon, partyRequestContext, ListCommunicationSessions and StartCommunicationAnalysis.

Conclusion The protection is useful as friction, not as authorization. Rewriting the same checks in C++ changes reverse-engineering cost but not the trust model.

F-06 Executable signing and metadata gaps LOW

The assessed executable is not Authenticode-signed and its standard file-version metadata is sparse. Signing will not prevent runtime patching, but it improves update authenticity, publisher identity and tamper detection around distribution.

F-07 Account-backed Community Filters can be reconstructed locally MEDIUM

The Community Filter catalog is unavailable to the isolated profile through the normal account-backed call, but an owner-authorized authenticated run returned 37 summaries and complete PSN Party and PSN Tunnel rule bodies. Once delivered to a client, those rule definitions are recoverable and can be translated into local custom filters without preserving an account session.

- Recovered PSN Party rule: UDP 1024-65535, length 96-140, two fixed payload prefixes.
- Recovered PSN Tunnel rule: UDP 50000-65535, length 96-158, six fixed payload prefixes.
- The PoC exposes the 37-name catalog offline but refuses to activate the 35 entries whose full rule bodies were not recovered.

Impact Client-delivered paid filter definitions cannot be kept secret by obfuscation alone. Signed short-lived manifests and server-side entitlement enforcement can control use, but the client must be assumed capable of reading any rule it receives.

Party Chat Puller and username-resolution analysis

Terminology The Party Chat screen labels the operation 'IP DECRYPTION.' Based on the recovered models, it is primarily a party-member-to-IP correlation workflow, not cryptographic decryption of a username from an IP address.

1. ListCommunicationSessions obtains PSN parties available to the linked account.
2. GetCommunicationSession returns a party roster with member identifiers and display names.
3. StartCommunicationAnalysis enters or monitors the selected session and consumes Party gateway events such as member_joined and remote_target.
4. CommunicationResult correlates memberId and displayName with ip, geoInfo and discoveredAt.

5. The UI displays other players and any discovered addresses; it can copy the correlated results.

A separate client path exposes DecryptPlayStation and ResolvePlaystationUsername. That path calls the OctoSniff API/resolver and is distinct from the Party Chat roster-and-address correlation. Recreating the Party Chat UI and client-side model is feasible from the recovered assets. Recreating a fully functional Puller still requires valid PSN authorization and either compatible gateway behavior or a replacement service implementation. No reconstructed client can reveal a peer IP when PSN uses Server Relay because the peer connection is absent.

Recreation assessment The interface, state machine and member-to-address correlation can be reproduced. There is no hidden username-decryption cipher in this path: display names come from the PSN roster, while the Puller waits for direct remote-target/address events. Relay mode is a hard topology limit, not an encryption challenge.

Packet-filter distinction The recovered PSN Party and Tunnel payload signatures can identify candidate traffic, but they do not derive a username. A visible PlayStation display name originates in an authorized party roster or resolver response and is then correlated with observed network events.

Recommended remediation

1. Move the paid decision to the server. Every Party HTTP request and event stream must carry subscriber-specific authorization.
2. Have the primary OctoSniff backend issue a signed, short-lived capability only after checking the authoritative subscription record.
3. Bind the capability to user ID, PSN account ID, device/session, audience, nonce, expiration and narrow scopes such as party:list, party:read and party:analyze.
4. Make the Party VPS reject unpaid, expired, revoked, replayed or incorrectly scoped capabilities before performing PSN work.
5. Rotate the recoverable shared gateway authentication key after the new enforcement is deployed.
6. Retain local checks only for navigation and messaging; never treat them as security controls.
7. Sign production binaries and updates. Strip symbols and reduce runtime metadata only as secondary hardening.

Required server-side test matrix

Case	Credential state	Expected	Enforcement point
Unpaid	Valid OctoSniff login, no add-on	403	Party VPS before PSN call
Expired	Capability past exp	401/403	Signature/token validator

Case	Credential state	Expected	Enforcement point
Cancelled	Subscription revoked	403	Revocation/current-state check
Replay	Nonce previously consumed	403	Replay cache
Modified	Scope/user/PSN changed	401	Signature validation
Wrong audience	Token minted for another service	401	Audience validation
Paid	Current add-on and valid PSN	200 + stream	Normal Party workflow

PoC package and reproducibility

The deliverable under OctoSniff-Security-Assessment\poc is intentionally application-specific. It is not a general process patcher and refuses any target whose SHA-256 differs from the assessed build. Its copied executable, process and visible window are renamed sigh_Divine sniff as a clear assessment marker. The launcher only closes an existing process at that exact isolated path, redirects settings and WebView data to package-local APPDATA and LOCALAPPDATA paths, and does not stop or modify the installed application. Closing the process removes all in-memory changes.

Artifact	Purpose
sigh_Divine sniff-Security-PoC.exe	Build-specific launcher and runtime patch verifier
app\sigh_Divine sniff.exe	Renamed, isolated byte-identical copy of the assessed target
src\OctoSniffSecurityPoc.cs	Reviewable PoC source
poc-run.log	ASLR-specific addresses and before/after patch evidence
Start-PoC.cmd	Convenience launcher
Stop-PoC.cmd	Stops the isolated PoC through a policy-safe wrapper
Stop-PoC.ps1	Stops only the isolated PoC executable path
offline-data\	37-name catalog plus exact PSN Party and Tunnel local rules
branding\	Red-slashed header, PNG icon and launcher ICO
How Access Worked\	Developer patch, validation and remediation handoff
profile\	Runtime-only isolated settings/WebView data; excluded from distribution
POC-ONLY.txt	Distribution and limitation notice

Limitations

- The controlled live run used Server Relay, so it could not demonstrate a positive peer-IP discovery result.
- The PoC deliberately skips the OctoSniff account lookup and does not create a valid backend account session; server APIs that independently validate a session can still reject it.
- The final account-free Party Chat state is an offline diagnostic with an empty party list; it does not claim or simulate authorized Sony access.
- Only PSN Party and PSN Tunnel have complete reconstructed rules. The other 35 Community Filter entries are metadata-only.

- The VPS source code and subscription database were not available; the absence of a subscriber assertion is based on recovered client protocol and call flow.
- No live credentials, PSN tokens, gateway keys, member names or IP addresses are included in this report.

Evidence integrity

Artifact	SHA-256
Assessed executable renamed sigh_Divine sniff.exe	2CF7F902FEA4E2109210CA36BE8ADE859B0CB7FB1ACEE4794B34AC08D785A317
sigh_Divine sniff PoC launcher	36E9CDEBA045615B638BEC35294DB677332C165E045869BDD5F0737CB9A7BF10
Recovered frontend	706974888F18A72F07C1E9A27A49D043528F3868C3204DAEA4FA20ECAC85E242
Account-free Party Chat screenshot	3129B6FB1396579DF1AC27050E7F7DF772BA2B881B096DA5F3FFFDADF3F9838B
Offline 37-entry catalog screenshot	0EC203ED942800D188754A2C927F3C5A6D9ED3C2F16ACB8298310B17AF616590
Controlled live result screenshot	ADA928D6542E2FE71D6AF7A7DD9A2F59316150F0B128F22B5E33841AE1A30E41
Cold-start interface selection screenshot	8655D8E2886EB489B8B728FEC05B7B772A8668CE2F3CBD9025382ABA1DF1A9E2
PoC runtime log	70E9213DC91606F8D343B89E4A015BBB8AD0F963AE4C593DAC1B3E6854542894

ASLR-specific addresses in the runtime log are evidence for that execution only. The launcher resolves functions from Go runtime metadata on each run rather than depending on fixed virtual addresses.